

System Design Document (SDD)

프로젝트 제목 : H.E.R.A Bot(구조현장지원 로봇)

버전 : 1.4

날짜 : [25.11.21]

1. Overview

H.E.R.A(Human Emergency Response Assistant) Bot은 두 대의 AMR이 지정된 구역을 순찰하다 응급 상황('fall-down' 상태)을 최초로 감지한 로봇이 즉시 해당 작업자에게 접근하고 다른 로봇이 구조 물품 조달 작업을 실행하는 로봇으로 전환되도록 설계되었다. 이를 통해 탐지·신고·물품 조달로 이어지는 응급 대응 절차를 자동화하여 초기 대응 속도와 안전성을 향상시킨다.

2. System Architecture

이 시스템은 두 대의 AI 기반 자율 주행 로봇(AMR)을 기반으로 구성되며 각 로봇은 응급 상황('fall-down' 상태)을 최초로 탐지한 로봇을 기준으로 '응급 상황 탐지 로봇'과 '구조 물품 조달 로봇'으로 구분된다.

응급 상황 탐지 로봇은 지정된 구역을 순찰하면서 응급 상황('fall-down' 상태)의 작업자를 탐지하여 접근 후 안전 관리자에게 알림을 보냄과 동시에 알람을 울리는 기능을 수행한다. 구조 물품 조달 로봇은 지정된 구역을 순찰하다 응급 상황 탐지 로봇이 'fall-down'상태의 작업자를 발견하면 그 알림을 받고 구조 물품 스테이션으로 이동 후 소리알람을 울리고 구조 물품을 응급 상황 위치로 조달하는 역할을 담당한다.

Monitoring System(PC3)은 탐지된 'fall-down' 작업자 정보를 기반으로 각 AMR에 할당된 PC(PC1, PC2)를 통해 센서 및 영상 데이터를 실시간 처리한다. 처리된 정보는 Wi-Fi와 ROS2 Fast DDS 통신을 통해 각 AMR의 Raspberry Pi로 전달되면 Raspberry Pi는 해당 제어 명령을 받아 로봇의 주행 및 안내 동작을 수행한다.

이러한 구조는 각 로봇이 역할에 따라 독립적으로 판단하고 행동할 수 있도록 하여 전체 시스템의 안정성과 유연성을 크게 향상시킨다.

2-1. High-Level Architecture Diagram

* 전체 자료 별첨

3. Component Design

3-1. Hardware Components

A. PC - AMR 제어용 2대 , Detection 및 모니터링용 1대

- 1) OS : Ubuntu 22.04 LTS + ROS2 Humble
- 2) Network : Wi-Fi

B. 자율 이동 로봇 (AMR) - TurtleBot4 2대 (ID 5 & 6)

- 1) 프로세서 : Raspberry Pi 4B-4GB RAM
- 2) 운영 체제 : Ubuntu 22.04 LTS + ROS2 Humble
- 3) 센서 :
 - OAK-D Pro : 실시간 객체 인식 및 거리 측정
 - RPLIDAR A1M8: 주변 환경을 360°로 스캔하여 고정밀 지도 생성 및 자율 주행 경로계획에 필수적인 데이터 제공
 - IMU : 자율주행의 자세 안정성을 높이는 관성 센서
 - 초음파센서 : 초음파 반사 신호로 감지하여 충돌 방지를 보조하는 근거리 센서
- 4) Battery : 26Wh 리튬 이온 배터리 (최대 4시간 배터리 수명)
- 5) Status LED
 - a) MTR LED : 모터 드라이버 작동 여부 표시
 - b) COMM LED : 네트워크 통신 상태 표시
 - c) Wi-fi LED : Wi-fi 연결 상태 표시
 - d) Battery LED : Green(정상), Red(30% 이하), None(배터리 상태 미확인)
 - e) Power LED : 전원 공급 정상 여부 표시

C. Docking Station : 수동 개입 없이 AMR의 충전을 위한 자율 Docking Station

3-2. Software Components

A. PC Software

- 1) Python3 : AI 및 데이터 처리 작업을 위한 기본 프로그래밍 언어
- 2) ROS2 : AMR 제어 및 통신, 관리를 위한 주요 플랫폼
- 3) OpenCV : 이미지 처리 라이브러리
- 4) Ultralytics(YOLO11n-pose) : 객체 탐지 모델을 사용하여 실시간 인식 및 위치 검출
- 5) SQLite3 : 데이터 저장 및 기록 보관을 위한 경량 데이터베이스
- 6) Flask : 인터페이스 구현을 위한 백엔드 프레임워크

B. AMR Software

- 1) **Python3** : TurtleBot4의 동작 제어, 센서 데이터 처리를 위한 언어
- 2) **ROS2** : SLAM, Navigation, 센서 데이터 통합
- 3) **turtlebot4_bringup** : 센서 초기화, TF 변환, 시스템 전체 bring-up
- 4) **turtlebot4_navigation** : URDF, xacro 기반 로봇 모델 제공, robot_state_publisher 를 통해 TF 트리 브로드캐스트
- 5) **turtlebot4_viz** : Rviz2 구성 패키지
- 6) **rplidar_ros** : RPLIDAR 장치 드라이버
- 7) **depthai_ros_driver** : OAK-D RGB/Depth 카메라 드라이버

4. Data Flow Design

4-1. Data Collection

A. 휴대폰(편집)

- 7초짜리 동영상 15개 (standing 5개, leaning 5개, lying 5개)
촬영 후 420x570 으로 size 를 조절

B. extract_keypoints.py

- 1) 원본 동영상 15개 입력
- 2) YOLO11n-pose 로 프레임별 Pose 추론
- 3) Bounding Box(bbox) + 17 keypoint(x,y) + confidence 추출
- 4) 자세 Feature 분석을 위해 정규화된 CSV 데이터셋 생성

C. build_features.py

- 1) 자세 분석을 위해 Core/Knee/Ankle Feature 10개 계산
- 2) 단일 Feature Dataset(features_all.csv)을 생성

D. label_features.py

- 1) features_all.csv 에 포함된 video_name을 기준으로 자세(standing, leaning, lying) 을 자동 부여
- 2) threshold 기반 자세 분류에 활용할 수 있는 완전한 labeled dataset(features_all_labeled.csv)을 생성

E. analyz_features.py

- 1) labeled dataset(features_all_labeled.csv)을 기반으로 Feature 의 posture 별 분포를 Histogram 으로 시각화

F. threshold_seatch.py

- 1) Feature-wise threshold 탐색을 통해 4개 Feature 각각의 최적 임계값 찾기
- 2) Feature 임계값이 $y_spread_norm < 0.1633$, $torso_angle_deg > 32.5167$, $shoulder_hip_diff_norm < 0.0791$, $bbox_ratio > 1.6425$ 가 되면 lying 판단

4-2. Data Processing and Analysis

A. detect_target_5.py

- 1) CompressedImage, Depth Image, CameraInfo를 수신
- 2) YOLO Pose 기반 추론(Keypoint 추출)
- 3) Lying Pose lying score ≥ 4 판단
- 4) 사람의 골반 중심 좌표 정보 및 Depth 거리 정보 송신
- 5) /target_point_5 발행

B. patrol_5.py

PatrolStateMachine 노드는 세 가지 AMR 상태를 관리한다.

- 초기 상태 : `current_state = 1` : 정해진 경로를 따라 순찰 이동
- 배터리 흐름 : /battery_state 구독
배터리 20% 미만 : Dock
충전 중일 때, `<is_charging = True>` : 95% 이상 충전되면 undock
- 동시에 돌아가는 타이머 루프 : `patrol_loop`(2초마다),
`alert_loop`(0.5초 마다), `rescue_loop`(1초마다),
`battery_monitor_loop`(1초마다)

robot5 가 응급 상황을 감지했을 때,

- 1) /target_point_5 가 1초 안에 2번 발행되면 `nav_to_goal` 취소, ALERT 모드로 전환
- 2) ALERT 모드 : `current_state = 2` , `alert_loop()` 동작, /tf_target_point_5 발행
- 3) /return_to_patrol를 True로 수신했을 때 Patrol 모드로 전환

robot6 가 응급 상황을 감지했을 때,

- 1) /target_point_6 토픽을 수신하면 `nav_to_goal` 취소, RESCUE 모드로 전환
- 2) RESCUE 모드 : 다른 노드에서 발행되는 `turtlebot6`에 대한 명령어를 수행할 수 있도록 대기
- 3) /return_to_patrol를 True로 수신했을 때 Patrol 모드로 전환

C. nav_to_target_5.py

- 1) 노드는 /tf_target_point_5 토픽을 구독
- 2) 수신된 좌표(x,y)를 기반으로 Nav2 행동 서버에 보낼 Goal 구성
- 3) Nav2 Action Server을 연결하고 Goal을 전송

- 4) AMR이 목표 지점 도착 완료
- 5) /arrival_target_5<Bool(data=True)> 발행

D. beep_pub_target_5.py

- 1) /arrival_target_5<Bool(data=True)> 토픽을 구독
- 2) listener5_callback() 은 도착 상태 True/False 를 확인
- 3) True 일 경우 publish_sound() 함수를 호출하여 경보음 구성
- 4) /robot5/cmd_audio 토픽을 발행하여 경보음 출력

E. nav_to_station_6.py

- 1) PointStamped 메시지 타입으로 /tf_target_point_5 이 좌표를 수신하는 순간 구조 물품 station(사전에 정의된 좌표) 이동 명령을 시작
- 2) Nav2 goal 생성
- 3) Action Client를 통한 /robot6/navigate_to_pose 전송
- 4) 도착 성공 시 STATUS_SUCCEEDED 반환
- 5) /arrival_station_6<Bool(data=True)> 발행

F. beep_pub_station_6.py

- 1) /arrival_station_6<Bool(data=True)> 토픽을 구독
- 2) listener6_callback() 은 True인지 여부 확인
- 3) publish_sound() 함수를 호출하여 경보음 구성
- 4) /robot6/cmd_audio 토픽을 발행하여 알림음 출력
- 5) 약 4초 후, /beep_finished_6 (Bool(data=True)> 발행

G. nav_to_sta_target_6.py

- 1) Fall down PointStamped 메시지 타입으로 /tf_target_point_5 이 좌표를 수신 및 저장
- 2) /beep_finished_6<Bool> 알람 종료 신호 수신 후 응급 상황 객체 좌표로 이동
- 3) Nav2 Action Client를 활용한 이동 목표 전송
- 4) Goal 완료 처리 후 응급 상황 객체에게 도착

H. system_monitor.py

- 1) /robot5&6/oakd/rgb/image_raw/compressed 를 수신 후 cv_bridge를 통해 OpenCV 이미지로 변환하며 최신 프레임은 shared buffer(latest_frames)에 저장 후 디스플레이
- 2) target_point_5&6 로 AMR이 탐지한 응급 상황 좌표를 수신하여 TimeStamped를 사용해서 변환해 event_time으로 관리
- 3) PointStamped 메시지가 도착하면 SnapShot을 생성하여 경로에 저장
- 4) SnapShot을 base64로 변환하여 SQLite DB에 AMR ID, event_time, 응급 상황 좌표, end_time, snapshot 경로 등 이벤트 로그를 저장

- 5) 안전 관리자가 PC에 뜨는 팝업을 확인하면(감지된 응급 상황 종료 시) AMR은 기존 순찰 모드로 자동 복귀
/return_to_patrol<Bool(data=True)>
- 6) /robot5&6/battery_state 를 웹 대시보드에 표시

4-3. Data Storage

- A. **PC 로그 저장** : 응급 상황 알림 팝업 및 로그가 모니터링 PC의 SQLite3에 저장되어 검토할 수 있다.

4-4. Alerting and Notification

- A. **실시간 경고 전송** : 응급 상황이 감지되면 AMR은 Wi-Fi를 통해 모니터링 PC로 알림을 전송한다.
- B. **PC 대시보드 표시** : PC 대시보드에서 보안 담당자는 실시간 비디오 피드, 알림 팝업 및 AMR 배터리 상태를 확인할 수 있다.

5. Detailed Design

5-1. AMR Navigation

- A. **네비게이션** : ROS2 와 LiDAR 기반의 SLAM을 활용하여 AMR이 생성된 지도 내에서 정해진 경로로 순찰한다.
- B. **Communication Protocol** : AMR은 Wi-Fi를 통해 모니터링 PC와 통신하며 응급 상황 탐지 및 작업 공간을 실시간으로 전송한다.

5-2. Detection

- A. **응급 상황 탐지** : Detection PC 에서 실행되는 YOLO11n-pose 기반 객체 탐지를 통해 작업자의 자세 변화를 분석
 - 자세 변화 분석 방법
 - a) Core features (y_spread_norm, x_spread_norm, bbox_ratio, torso_angle_deg, shoulder_hip_diff_norm, head_y_norm), Knee features(knee_avg_y, knee_height_diff) , Ankle features(ankle_spread_dx, ankle_avg_y) 계산 및 분석
 - b) 실제 분류 모델(threshold_search.py) 에서 핵심 feature 4개 (y_spread_norm, torso_angle_deg, shoulder_hip_diff_norm, bbox_ratio)

- c) standing에서 lying 전환 판단 기준 ('fall-down' 상태) :
 $y_spread_norm < 0.1633$, $torso_angle_deg > 32.5167$,
 $shoulder_hip_diff_norm < 0.0791$, $bbox_ratio > 1.6425$

5-3. Monitoring System

모니터링 시스템은 PC3로 진행되며 AMR의 센서 데이터 처리, 객체 판별 및 탐지, 상태 모니터링 및 데이터 베이스 관리를 담당한다.

- A. 작업자 상태 확인 및 상황 종료 후 순찰 모드 복귀: 작업 공간을 실시간 카메라 피드를 표시하며 응급상황이 발생하면 안전 관리자 PC에서 알람이 뜨고 응급 상황이 종료된 후 AMR이 기존 순찰 모드로 전환하도록 한다.
- B. AMR 상태 확인 : 실시간으로 AMR의 배터리 상태를 확인한다.
- C. 데이터 관리 및 저장 : Flask 기반 웹 대시보드를 제공하여 순찰 구역별로 SQLite DB에서 이벤트 로그(AMR ID, 시간, 좌표, 상황 종료시간, 이미지 경로)를 통합하여 관리한다.

6. Security Design

- A. 데이터 암호화 : AMR 과 모니터링 PC 간 통신은 Fast DDS 의 DDS Security Plugin을 사용하여 AES-GCM 기반 전송 데이터 암호화를 적용한다.
- B. User name/Password : Monitoring System 내 로그인 단계에서 비밀번호는 안전하게 관리되어야 하며 주기적인 갱신이 요구되어야 한다. 잘못된 인증 정보 사용 시 연결을 즉시 거부하며 정상적인 인증 정보의 재입력을 요구한다.
- C. 장치 보안 강화 : AMR의 Raspberry Pi 4B는 SSH Key 기반 인증 및 OS 보안 패치를 유지할 통해 AMR 내부 컴퓨팅 장치의 안정성을 강화한다.
- D. 네트워크 보안 : AMR 전용 Wi-Fi SSID 를 구성하고 WPA2-PSK 또는 WPA3-SAE 암호화를 적용하여 외부망 접근을 차단한다.

7. Performance Requirements

- A. 시스템 가동률 :** AMR은 최소 2시간 이상 연속 운영이 가능해야 하며 충전 배터리가 20% 이하로 감소할 경우 Docking Station으로 복귀할 수 있어야 한다.
- B. 확장성 :** 2대의 AMR 운영에 맞춰 설계되었으나 추가 유닛이 필요할 경우, 최소한의 Architecture 변경으로 지원 가능하다.
- C. AMR 이동 :** AMR이 사전에 지정된 waypoint 좌표, target_point 좌표에 정확히 도달해야 하며 도착 시 위치 오차는 $\pm 0.2m$ 이내로 유지되어야 한다.
- D. AMR Localization :** AMR은 초기 pose 설정 이후 지속적으로 자신의 위치를 안정적으로 추정해야 하며 SLAM으로 생성된 Map과의 정확성이 유지되어야 한다.
- E. AMR SLAM + Navigation :** AMR은 SLAM 기반으로 실시간 Map을 구성하며 목적지까지 자율 주행이 가능해야 한다.
- F. 객체 인식 :** 객체를 RGB 카메라 기반 YOLO11n-pose를 통해 Confidence 0.75 으로 안정적으로 인식해야 한다.
- G. 모니터링 :** 응급 상황이 감지된 후 안전 관리자 PC(Flask Dashboard)에 응급 상황 발생 팝업이 표시되기까지의 시간은 5초 이내여야 한다.
- H. 알림음 반응 시간 :** AMR이 지정된 waypoint 좌표 또는 구조 물품 Station에 도착했을 때, 알림음 반응 시간은 5초 이내여야 한다.

8. Error Handling and Recovery

- A. 네트워크 손실 :** AMR1 또는 AMR2가 구동 중 Wi-Fi 연결이 끊기면 각 AMR과 연결된 PC로 ping을 확인한 후 연결이 복원되면 데이터 전송을 재개해야 한다.
- B. 하드웨어 오류 :** AMR은 정기적으로 센서 및 하드웨어 상태를 점검한다.
- C. 배터리 부족 :** 배터리가 20% 이하일 경우 AMR은 자율적으로 Docking Station으로 복귀해야 한다.

9. Testing and Validation

A. 단위 테스트 : ROS2 Node 및 Flask 서버를 포함한 모든 소프트웨어 구성요소를 개별적으로 테스트하여 기능을 검증한다.

- 1) **Detection TEST** : YOLO11n-pose 기반 모델을 사용, AMR을 통해 Standing Person과 Lying Person을 판별 및 감지 여부를 확인한다.
- 2) **System Monitoring TEST** : 응급 상황이 발생한 즉시 안전 관리자 모니터에 알림이 뜨고 화면 내에서 응급 상황을 감지한 로봇의 아이디, 상황 발생 시간, 좌표 등 로그 및 이미지, 영상을 확인한다.
- 3) **Nav_to_station TEST** : AMR이 지정된 Station으로 이동해서 도착 후 알람 기능을 실행한다.
- 4) **Nav_to_target TEST** : 지정된 구조 물품 Station에서 응급 상황이 발생한 위치 좌표로 이동해서 알람 기능을 실행한다.
- 5) **AMR Auto-Charging TEST** : AMR의 배터리가 20% 이하가 되면 Charging Station으로 자율 이동하여 95%까지 충전한 뒤 다시 복귀한다.

B. 통합 테스트 : 모든 PC와 AMR을 통합하여 원활한 데이터 흐름과 명령 실행을 보장한다.

C. 현장 테스트 : 지정된 구역에서 AMR을 테스트하여 자율 주행, 응급 상황 탐지 및 알람 기능을 실제 조건에서 검증한다.

10. Deployment and Maintenance Plan

A. 배포 단계

- 1) PC에 ROS2, SQLite Viewer, Flask를 설치한다.
- 2) 모니터링 대시보드를 구성하고 AMR을 Wi-Fi로 연결한다.

B. 유지보수 일정

- 1) 매월 AMR과 PC의 소프트웨어 업데이트로 보안 패치 적용
- 2) 분기마다 AMR의 하드웨어 점검, 센서 보정 및 배터리 검사

11. Appendix

A. 라이브러리 및 종속성 : ROS2, OpenCV, YOLO, Flask, SQLite3 등 사용된 소프트웨어 종속성 목록

B. 용어 사전 : AMR, SLAM, YOLO 등 주요 용어의 정의

- C.** 참고 문헌 : ROS2, WiFi, Raspberry Pi, TurtleBot4 관련 기술,
Mali, V., & Jaiswal, S. (2025). *Pose-Based Fall Detection System:
Efficient Monitoring on Standard CPUs*. arXiv preprint
arXiv:2503.19501